VOLUME I · EXAM CHEATSHEET · STUDY GUIDE · ALL FOUR MODULES

CMS 710 — Principles of Programming Languages

PGD Computer Science, Rivers State University, Nkpolu-Oroworukwo, Port Harcourt · built from the four class modules of the lecturer and the lecturer, with the comparison tables and code examples reconstructed from the source PDFs

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 12 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturers:** the lecturers

WHAT THE TEST QUESTIONS TELL YOU

Four questions set by **the lecturer** are on file, and **all four come from Module 4 alone**:

1. Define **control flow**; explain the difference between **sequence and iteration**
2. What is **recursion** and how does it differ from **iteration**, with an example
3. Differentiate **global and local scope**, and use that to explain **variable lifetime**
4. Compare **pass by value and pass by reference**, with an example

That is a strong signal. **Module 4 is the highest-yield module on the course** — learn §5 of this volume first. All four are answered in full in Volume II.

Two more things the questions reveal. Every one says "**differentiate**", "**compare**" **or** "**with an example**" — so **answer in a table and always append a code example**, in any of Python, JavaScript or C++. And each is a **pair** of concepts: this examiner tests boundaries between things, not isolated definitions.

Good news on workload: Module 3 (arrays, lists, stacks, queues, trees) is the same ground as **CMS 702**, and Module 4 overlaps heavily with **CMS 708**. Two exams you have already prepared for cover half of this one.

1 Definitions to write word-for-word

TERM	DEFINITION
Programming language	A **formal language consisting of a set of symbols, keywords, syntax rules and semantic rules** used to communicate instructions to a computer; the means through which programmers express algorithms that can be translated into machine-executable instructions.
Syntax	The **grammatical rules governing how program statements are written** — it specifies the **legal structure** of programs.
Semantics	The **meaning associated with syntactically correct statements**. **Syntax determines structure; semantics determines meaning.**
Language specification	A **formal document describing all aspects of a programming language** — syntax, semantics, data types, operators, control structures, libraries and runtime behaviour. It is the authoritative reference for implementation and usage.
Grammar / BNF	A formal description of **how valid program constructs can be formed**. The most common notation is **Backus–Naur Form**, e.g. `<assignment> ::= <identifier> = <expression>`.
Data type	A **classification specifying the kind of values that can be stored, the operations permitted on them, the memory required, and the interpretation of the stored data**. It defines **a set of values and a set of permissible operations**.
Type system	A **set of rules governing how data types are defined, checked and used** in a language, determining what values variables may contain, what operations are permitted and how errors are detected.
Static typing	Type checking performed **during compilation**.
Dynamic typing	Type checking performed **during program execution**.

TERM	DEFINITION
Type safety	The **extent to which a language prevents invalid operations**.
Data abstraction	The process of **hiding implementation details while exposing only essential features**, so programmers focus on **what** something does rather than **how** it is implemented.
Data structure	A **method of organizing and storing data in a computer so that it can be accessed and modified efficiently**.
Abstract Data Type	A **logical description of a data structure specifying the data it stores and the operations that can be performed, without specifying implementation details** — what the structure does, not how it is implemented.
Parse tree	A tree that **represents the grammatical structure of a program according to the language grammar**; generated during **syntax analysis**.
Abstract Syntax Tree	A **simplified version of a parse tree** that **removes unnecessary grammar details while preserving program meaning**.
Control structures	The **mechanisms that determine the order in which operations are executed**; they govern flow of execution, decision-making, repetition and how functions interact.
Control flow	The **order in which statements, instructions or function calls are executed** in a program.
Recursion	A **control mechanism in which a function calls itself to solve a problem**, consisting of a **base case** that terminates it and a **recursive case** that calls itself with a smaller problem.
Scope	**Where a variable can be accessed within a program** — it controls variable **visibility and lifetime**.
Variable lifetime	The **period during which a variable exists in memory**. It depends on **scope, storage allocation method and program execution state**.
Data flow	**How information moves through a program** — between variables, functions, modules and program components.
Structured programming	A paradigm emphasizing **sequence, selection and iteration** while **avoiding uncontrolled jumps** such as excessive use of `GOTO`.

2 Module 1 — language definition structure

THREE PERSPECTIVES ON A PROGRAMMING LANGUAGE

1. **Communication perspective** — a medium between the programmer and the computer
2. **Problem-solving perspective** — tools and constructs for solving computational problems
3. **Formal system perspective** — a mathematically defined system governed by precise syntactic and semantic rules

EVOLUTION — THE FOUR GENERATIONS

GEN	TYPE	EXAMPLE	CHARACTERISTICS
1GL	Machine language	`10110000 01100001`	Machine dependent · difficult to understand · **fast execution**
2GL	Assembly language	`MOV AX, 5 ADD AX, 2`	Symbolic instructions · **requires an assembler** · hardware dependent
3GL	High-level	C++, Java, Python, JavaScript	**High-level abstraction** · portable · easier development
4GL	Logic / AI-oriented	Prolog	Logic-based · knowledge representation · AI applications

SYNTAX VS SEMANTICS — THE CLASSIC PAIR

```
SYNTAX  — is it legally written?
   Correct:    x = 10
   Incorrect:  = x 10        ← violates the rules

SEMANTICS — what does it mean?
   x = 10 + 20
   Syntax says: correctly written.
   Semantics says: add 10 and 20, store in x.
```

> **The one-liner: syntax determines structure, semantics determines meaning.** A statement can be syntactically perfect and semantically meaningless.

THE FIVE GOALS OF LANGUAGE DESIGN

GOAL	DEFINITION	ACHIEVED THROUGH
Readability	The ease with which programs can be **understood**, including code written by others	Simplicity · consistency · clear syntax · meaningful keywords
Writability	The ease with which programmers can **create** programs	Powerful constructs that reduce development effort
Reliability	The ability of a program to **perform according to its specification** under various conditions	Strong type checking · exception handling · restricted operations
Maintainability	The ease with which software can be **modified, corrected or extended**	Modular design · clear documentation · consistent coding standards
Efficiency	The effective **utilization of system resources**	CPU time · memory · storage — why C++ suits performance-critical work

THE THREE PRINCIPLES OF LANGUAGE DEFINITION

1. **Formal rules govern every language.** Languages are not arbitrary collections of symbols; without precisely defined rules for syntax, semantics and execution, compilers and interpreters could not process programs consistently.
2. **Simplicity versus expressiveness.** Simple languages give easier learning and better readability but limited abstraction; expressive languages give powerful constructs and less code but increased complexity.
3. **Efficiency versus safety.** C++ emphasizes performance; Python emphasizes productivity and safety. This trade-off drives adoption in different domains.

BNF — THE NOTATION TO REPRODUCE

```
<assignment> ::= <identifier> =
                 <expression>
```

Meaning: an assignment statement consists of **an identifier, an assignment operator and an expression**. The statement `x = y + z` satisfies this rule.

The same program in three languages MODULE 1 EXAMPLE

PYTHON

```
age = 20

if age >= 18:
    print("Adult")
```

JAVASCRIPT

```
let age = 20;

if (age >= 18) {
    console.log("Adult");
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    if (age >= 18) {
        cout << "Adult";
    }
    return 0;
}
```

The observation to write: all three implement the **same logic**, but differ in **syntax, type systems, block structures and execution models** — and those differences **reflect distinct language design philosophies**.

PRINCIPLE	PYTHON	JAVASCRIPT	C++
Block structure	**Indentation**	Braces	Braces
Typing style	Dynamic	Dynamic	**Static**
Compilation model	Interpreted	JIT / interpreted	**Compiled**
Complexity	Low	Medium	High
Memory control	Automatic	Automatic	Manual / automatic
Readability	**Very high**	High	Moderate
Runtime speed	Moderate	Moderate	**High**

3 Module 2 — data types and type systems

WHAT A DATA TYPE SPECIFIES — FOUR THINGS

1. The **kind of values** that can be stored
2. The **operations** that can be performed on them
3. The **amount of memory** required for storage
4. The **interpretation** of the stored data

Formally, a data type defines **a set of values and a set of permissible operations**.

IMPORTANCE OF DATA TYPES — FOUR POINTS

1. **Improve program correctness** — they prevent invalid operations
2. **Improve reliability** — type checking detects errors before execution
3. **Improve efficiency** — the compiler can optimize memory allocation when types are known
4. **Enhance readability** — proper use makes programs easier to understand

THE THREE CATEGORIES OF DATA TYPE

CATEGORY	DEFINITION	EXAMPLES
Primitive	The **basic building blocks provided directly by the language**; they **cannot be decomposed** into simpler types	Integer · floating-point · character · boolean · string
Composite	**Combine multiple primitive values into a single structure**	Arrays · lists · tuples · dictionaries · structures · objects
User-defined	Types **created by the programmer**	Structures · classes · enumerations · records

USER-DEFINED TYPES IN THREE LANGUAGES

```
C++       struct Student {
              string name;
              int    age;
          };

Python    class Student:
              def __init__(self, name, age):
                  self.name = name
                  self.age  = age

JavaScript class Student {
               constructor(name, age) {
                   this.name = name;
                   this.age  = age;
               }
           }
```

STATIC VS DYNAMIC TYPING LIKELY QUESTION

BASIS	STATIC TYPING	DYNAMIC TYPING
When checked	During **compilation**	During **execution**
Languages	C++	Python, JavaScript
Advantages	Early error detection · improved reliability · better performance · compiler optimization	Greater flexibility · faster development · less code
Disadvantages	Reduced flexibility · more verbose code	**Runtime errors** · reduced type safety

```
C++    int x = 10;
       x = "CMS710";      ← COMPILATION ERROR

Python x = 10
       x = "CMS710"       ← perfectly legal
```

STRONG VS WEAK TYPING — A DIFFERENT AXIS

BASIS	STRONG TYPING	WEAK TYPING
Rule	**Prevents inappropriate type conversions**	**Allows implicit type conversions**
Example	Python: `age + "years"` → **error**	JavaScript: `"5" + 2` → **`"52"`**, the number is converted to a string

Do not confuse the two axes. Static/dynamic is about **when** types are checked; **strong/weak** is about **how strictly** conversions are enforced. Python is **dynamic but strong**; JavaScript is **dynamic and weak**; C++ is **static and strong**. That single sentence answers most of a question on this topic.

DATA ABSTRACTION

Definition: hiding implementation details while exposing only essential features, so programmers focus on **what** an object does rather than **how** it is implemented.

The car analogy from the notes: a driver uses the **steering wheel, accelerator and brake** without needing to understand the internal engine mechanisms. Similarly, programmers interact with data structures through **well-defined interfaces**.

Four benefits: simplicity (reduces complexity) · **reusability** (abstract components can be reused) · **maintainability** (changes to implementation do not affect users) · **security** (internal details remain hidden).

FEATURE	PYTHON	JAVASCRIPT	C++
Typing style	Dynamic	Dynamic	**Static**
Type safety	**Strong**	Weak / moderate	**Strong**
Compilation	Interpreted	JIT / interpreted	Compiled
Memory control	Automatic	Automatic	Manual / automatic
Flexibility	**Very high**	High	Moderate
Runtime speed	Moderate	Moderate	**High**

FLEXIBILITY VERSUS RELIABILITY — THE CENTRAL TRADE-OFF

Dynamic languages: rapid development and easier experimentation, at the cost of **increased runtime errors**.
Static languages: strong verification and improved reliability, at the cost of **more development effort**.
Performance follows the same line: static typing enables compile-time optimization, efficient memory allocation and faster execution, while dynamic typing introduces runtime type checking and additional overhead — **which is why C++ often outperforms Python and JavaScript in computationally intensive applications**.

4 Module 3 — data structures, lists and trees

> **Overlap alert.** Arrays, lists, stacks, queues and trees are the same material as **CMS 702**. What is **new here** and specific to this course is **§3.12–§3.14: parse trees, abstract syntax trees, and the role of trees in language processing**. If time is short, revise those and treat the rest as recall.

THE THREE PRINCIPLES OF DATA STRUCTURES

1. **Efficient organization of data** — searching an unsorted array may require scanning every element, while a binary search tree may require far fewer comparisons.
2. **Abstraction of complexity** — a programmer may use a stack without understanding its internal memory allocation.
3. **Trade-off between time and space** — improving speed often requires more memory; reducing memory may increase processing time.

ABSTRACT DATA TYPES

An **ADT** specifies **the data it stores and the operations that can be performed, without specifying implementation**. The **Stack ADT** declares `Push()`, `Pop()`, `Peek()` and `IsEmpty()` — and **does not specify** whether the stack is built from arrays, linked lists or dynamic memory.

Four advantages of ADTs: abstraction (reduces complexity) · **reusability** (can be implemented in multiple ways) · **maintainability** (implementation can change without affecting users) · **modularity** (encourages separation of concerns).

THE FOUR LINEAR STRUCTURES

STRUCTURE	DEFINITION AND RULE	REAL-LIFE USES
Array	Elements in **contiguous memory**, accessed by **index**. Fixed size, indexed access, efficient retrieval, homogeneous data	Student records
List	An **ordered collection** that, unlike an array, can **grow and shrink dynamically**	Any changing collection
Stack	**LIFO** — the last element inserted is the first removed. `Push`, `Pop`, `Peek`, `IsEmpty`	Browser history · undo · **function calls** · expression evaluation
Queue	**FIFO** — the first element inserted is the first removed. `Enqueue`, `Dequeue`, `Front`, `IsEmpty`	Bank customers · print queues · OS scheduling · network packets

Arrays: fast access and simple implementation, but **fixed size and costly insertion/deletion**. **Lists:** dynamic and flexible,

EACH STRUCTURE IN THREE LANGUAGES

ARRAY / LIST

```
Python      scores = [70, 80, 90, 85, 95]
JavaScript  let scores = [70, 80, 90, 85, 95];
C++         int scores[5] = {70, 80, 90, 85, 95};
            vector<string> students;    // dynamic
```

STACK

```
Python      stack = []
            stack.append(10); stack.pop()
JavaScript  let stack = [];
            stack.push(10); stack.pop();
C++         #include <stack>
            stack<int> s;
            s.push(10); s.pop();
```

QUEUE

```
Python      from collections import deque
            queue = deque()
            queue.append(10); queue.popleft()
JavaScript  let queue = [];
            queue.push(10); queue.shift();
C++         #include <queue>
            queue<int> q;
            q.push(10); q.pop();
```

BINARY TREE NODE

```
Python      class Node:
                def __init__(self, value):
                    self.value = value
                    self.left  = None
                    self.right = None
            root = Node(50)

C++         class Node {
            public:
                int   value;
                Node* left;
                Node* right;
                Node(int v) { value = v;
                              left = nullptr;
                              right = nullptr; }
            };
            Node* root = new Node(50);
```

A tree is a **hierarchical structure of nodes connected by edges** — root, parent, child, leaf. A **binary tree** has **at most two children** per node. Applications: searching, sorting, expression evaluation, database indexing.

but **extra memory overhead and slower than arrays** for some operations.

Parse trees, ASTs and language processing THE COURSE-SPECIFIC PART

PARSE TREE for the expression `a + b * c`

```
          +
         / \
        a   *
           / \
          b   c
```

The parse tree shows that MULTIPLICATION is evaluated BEFORE addition.

ABSTRACT SYNTAX TREE

```
Assignment
├── Variable(a)
└── Addition
    ├── b
    └── c
```

A **parse tree** represents the **grammatical structure of a program according to the language grammar** and is generated during **syntax analysis**. An **AST** is a **simplified parse tree** that **removes unnecessary grammar details while preserving meaning**. ASTs are used in **compilers, interpreters, static analysis tools, code optimization systems and modern AI code assistants**.

WHERE TREES SIT IN THE TRANSLATION PIPELINE

PHASE	PRODUCES
Lexical analysis	**Tokens**
Parsing (syntax analysis)	**Parse trees**
Semantic analysis	**Abstract syntax trees**
Code generation	Executable instructions, **from the AST**

The conclusion the module draws: trees are central to compiler and interpreter design. If a question asks why trees matter in a *programming languages* course rather than a data structures one, this table is the answer.

FEATURE	PYTHON	JAVASCRIPT	C++
Built-in list support	Excellent	Excellent	Vector library
Dynamic resizing	Yes	Yes	Yes
Memory management	Automatic	Automatic	Manual / automatic
Tree implementation complexity	**Low**	Moderate	High
Runtime performance	Moderate	Moderate	**High**
Ease of use	**Very high**	High	Moderate

5 Module 4 — control structure and data flow ALL FOUR TEST QUESTIONS

CONTROL FLOW — THE DEFINITION TO MEMORISE

Control flow refers to **the order in which statements, instructions or function calls are executed in a program**. It determines:

- Which instruction **executes next**
- Under what conditions execution **changes direction**
- How **repetition** occurs
- How **data moves** through program components

It is commonly represented using **flowcharts, control-flow graphs or execution traces**. **Control structures** are the mechanisms that govern it, and their design significantly influences **program readability, software reliability, maintainability and execution efficiency**.

THE THREE PRINCIPLES OF CONTROL STRUCTURES

1. **Program behaviour is determined by control structures** — the same data and operations may produce different results depending on the sequence and conditions under which they execute.
2. **Simplicity improves readability** — Python emphasizes readability through simplified control constructs.
3. **Abstraction reduces complexity** — higher-level abstractions hide low-level control details: **foreach loops, iterators, generators, recursive functions**.

THE FOUR KINDS OF CONTROL FLOW

KIND	WHAT IT DOES
Sequential	The **simplest form** — statements executed **one after another in the order they appear**. Simple, predictable, easy to understand, but **cannot support decision-making or repetition**
Selection	Choose among **alternative execution paths based on conditions** — this enables **decision making**
Iteration	Execute a set of instructions **repeatedly until a condition is satisfied** — commonly called **looping**
Recursion	A function **calls itself** to solve a problem

Selection has three types: single (execute a statement if a condition is true) · **double** (choose between two alternatives) · **multiple** (choose among several — the `switch`).
Advantages of selection: supports decision making · improves flexibility · enhances program intelligence.
Advantages of iteration: reduces code duplication · improves efficiency · simplifies repetitive tasks.

SEQUENCE VS ITERATION TEST Q1

BASIS	SEQUENCE	ITERATION
Definition	Statements executed **once each, in written order**	A block executed **repeatedly** until a condition is satisfied
Repetition	**None**	**Yes** — that is its purpose
Condition	No condition involved	**Controlled by a condition**
Order	Strictly top to bottom	Returns to the start of the block
Constructs	Ordinary statements	`for` · `while` · `do–while`
Purpose	Perform steps in order	Avoid rewriting the same code

```
SEQUENCE                ITERATION
x = 10                  for i in range(5):
y = 20                      print(i)
z = x + y
print(z)                prints 0 1 2 3 4
                        — one statement, five executions
Executes 4 statements,
each exactly once.
```

THE THREE LOOPS

LOOP	USED WHEN
`for`	The **number of repetitions is known**
`while`	Repetitions **depend on a condition**
`do–while`	**Executes at least once** before testing the condition

```
Python      for i in range(5):
                print(i)

JavaScript  for (let i = 0; i < 5; i++) {
                console.log(i);
            }

C++         for (int i = 0; i < 5; i++) {
                cout << i;
            }

Python      count = 1
            while count <= 5:
                print(count)
                count += 1
```

Recursion TEST Q2

Recursion is a control mechanism in which **a function calls itself** to solve a problem. A recursive solution consists of a **base case**, which terminates the recursion, and a **recursive case**, which calls itself with a smaller problem.

$$n! = n \times (n - 1)!$$

Python

```python
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

JavaScript

```javascript
function factorial(n) {
    if (n === 1) { return 1; }
    return n * factorial(n - 1);
}
```

C++

```cpp
int factorial(int n) {
    if (n == 1) return 1;
    return n * factorial(n - 1);
}
```

ADVANTAGES	DISADVANTAGES
Elegant solutions	**Higher memory consumption**
Natural representation of hierarchical structures	**Risk of stack overflow**
Useful for trees and graphs	**Often slower than iteration**

The sentence that earns the comparison mark: recursion and iteration can solve the same problems, but **iteration repeats a block within one function call using a loop**, while **recursion repeats by making new function calls, each adding a stack frame**. That difference is why recursion costs more memory and risks stack overflow, and why it reads more naturally for **trees and hierarchical data**. Full comparison table in Volume II.

Scope and lifetime TEST Q3

Scope determines where a variable can be accessed within a program. It is fundamental because it **controls variable visibility and lifetime**.

TYPE OF SCOPE	ACCESSIBLE	EXAMPLE
Global scope	**Throughout the program**	Python: `x = 10` at module level
Local scope	**Only within a function or block**	Python: `y = 20` inside `def test():`
Block scope	Only **inside a specific block**	JavaScript: `let age = 20;` inside `if (true) { }` — reading `age` outside **produces an error**

Importance of scope — four points: prevents naming conflicts · enhances security · improves maintainability · supports modular design.

Variable lifetime refers to **the period during which a variable exists in memory**. A variable's lifetime depends on **scope, storage allocation method and program execution state**.

- **Global variables** typically exist **throughout execution**.
- **Local variables** exist **only while their function executes**.

How the two questions link — this is exactly what test Q3 asks. Scope is about **where** a variable is visible; lifetime is about **when** it exists. They are connected because **the scope a variable is declared in determines how long it lives**: a variable in global scope lives for the whole program, while one in local scope is created when the function is entered and destroyed when it returns. **Scope is spatial, lifetime is temporal.**

Parameter passing and data flow TEST Q4

MECHANISM	HOW IT WORKS
Pass-by-value	A **copy** of the argument is passed. **Changes inside the function do not affect the original variable**
Pass-by-reference	The function receives a **reference to the original variable. Changes affect the original**
Pass-by-object-reference	Python's approach — **objects are passed by reference to object values**, so appending to a list inside a function **modifies the original list**

DATA FLOW

Data flow refers to **how information moves through a program** — between **variables, functions, modules and program components**. Most programs follow the **Input → Processing → Output** model:

```
C++ pass-by-value          C++ pass-by-reference
void increment(int x) {    void increment(int &x) {
    x++;                       x++;
}                          }
original unchanged         original modified

Python pass-by-object-reference
def modify(lst):
    lst.append(100)      ← the original list IS modified
```

```
number = int(input())
square = number * number
print(square)

    User Input
        ↓
     Variable
        ↓
    Processing
        ↓
      Output
```

IMPERATIVE VS FUNCTIONAL CONTROL FLOW

BASIS	IMPERATIVE	FUNCTIONAL
Focus	**How** tasks should be performed	**What** should be computed
Example	`total = 0 for n in numbers: total += n`	`total = sum(numbers)`
Detail	Every step is written out	Implementation details are **hidden**
Languages	C++, JavaScript, Python	Python and JavaScript support functional style

Advantages of the functional approach: simpler code · improved abstraction · **reduced side effects**.

FEATURE	PYTHON	JAVASCRIPT	C++
Readability	**Very high**	High	Moderate
Block structure	Indentation	Braces	Braces
Recursion support	Excellent	Excellent	Excellent
Functional features	Moderate	**High**	Moderate
Performance	Moderate	Moderate	**High**
Parameter passing complexity	**Low**	Moderate	High

LANGUAGE DESIGN PERSPECTIVE — A READY-MADE CLOSING PARAGRAPH

Control structures reveal differences in language philosophy. **Python** emphasizes **simplicity, readability and reduced syntax**; **JavaScript** emphasizes **flexibility, event-driven programming and functional constructs**; **C++** emphasizes **performance, explicit control and system-level programming**. These differences demonstrate how language designers **balance readability, abstraction, flexibility and efficiency**.

Structured programming, in this module's words, emphasizes **sequence, selection and iteration** while **avoiding uncontrolled jumps such as excessive use of GOTO**. Its benefits: **improved readability** (programs easier to understand) · **improved reliability** (errors easier to identify) · **easier maintenance** (modifications simpler).

6 Things you can lose easy marks on

1. Defining syntax and semantics with the same words. **Syntax = structure, semantics = meaning.**
2. Confusing the two typing axes. **Static/dynamic = when checked; strong/weak = how strictly enforced.** Python is
7. Answering "differentiate" in prose. **This examiner's questions all say compare or differentiate — use a table.**

dynamic *and* strong.

3. Saying a parse tree and an AST are the same. The **AST is the simplified one**, with grammar details removed.
4. Calling C++ interpreted or Python compiled. C++ is **compiled**, Python **interpreted**, JavaScript **JIT/interpreted**.
5. Giving stacks FIFO or queues LIFO. **Stack = LIFO, queue = FIFO.**
6. Describing a data type as only "a kind of value" — it is **a set of values *and* a set of permissible operations**.
8. Omitting the example when the question says "with an example". Q2 and Q4 both demand one.
9. Treating scope and lifetime as the same thing. **Scope is spatial (where), lifetime is temporal (when).**
10. Saying recursion is always better. It is **elegant but costs memory, risks stack overflow and is often slower**.
11. Forgetting that a recursive function needs a **base case** — without it, infinite recursion.
12. Writing only C++. The course compares **Python, JavaScript and C++** throughout; a comparison answer that names all three scores higher.

TIMING PLAN FOR A 3-HOUR PAPER

Read everything first. Because every question on this course is a **comparison or definition**, the fastest marks come from **drawing the table first and filling it in** — a table with six bases of comparison is worth more than three paragraphs and takes half the time. Budget **30 minutes per question**, and for each one: **define both terms (2 sentences) → table → one code example → one closing sentence on why the difference matters**. That four-part shape fits every question this examiner has set. Leave the code examples until the table is done, so that if time runs out you have lost the cheapest part rather than the dearest.